

Lecture Notes:

Recurrent Neural Networks

CIC1205 - Machine Learning

Eduardo Bezerra

June 5, 2026

0 Contents

Before You Read	3
Learning Objectives	3
1 Introduction	4
1.1 Why Not Just Use an MLP or CNN for Sequences?	4
1.2 The Sequential Inductive Bias	4
1.3 Conceptual Roadmap	5
2 The Vanilla RNN	6
2.1 State Update Equations	6
2.2 Unrolling Through Time	7
2.3 A Worked Example	8
2.4 Counting Parameters	8
3 Training RNNs: Backpropagation Through Time	9
3.1 The BPTT Algorithm	9
3.2 The Vanishing and Exploding Gradient Problems	9
3.3 Truncated BPTT and Gradient Clipping	10
4 Long Short-Term Memory (LSTM)	11
4.1 Motivation and Architecture Overview	11
4.2 LSTM Equations	11
4.3 Understanding the Gates	12
4.4 Counting LSTM Parameters	13
5 Gated Recurrent Unit (GRU)	15
5.1 Simplifying the LSTM	15
5.2 GRU Equations	15
5.3 Interpreting the GRU Gates	15
6 Sequence Modeling Tasks and RNN Configurations	17
6.1 Taxonomy of Sequence Tasks	17
6.2 Bidirectional RNNs	17
6.3 Stacked (Deep) RNNs	18
7 Implementation in PyTorch	19
7.1 RNN Modules in PyTorch	19
7.2 Sequence Classifier	19
7.3 Character-Level Language Model	20
7.4 Training Loop with Gradient Clipping	21
8 Common Pitfalls and Debugging RNNs	22
8.1 First Sanity Checks	22
8.2 Debugging Checklist	23
9 Connection to the Broader Deep Learning Landscape	23
9.1 RNNs as Structured MLPs	24

9.2 The Transformer and the Limits of RNNs	24
10 Final Remarks	25
10.1 Summary of Key Concepts	25
10.2 Recommended Next Steps	26

0 Before You Read

These notes build directly upon the MLP and CNN lecture notes. The following concepts are assumed to be already understood. Review them if necessary before proceeding.

- The MLP architecture: layers, neurons, weight matrices, and bias vectors.
- The forward pass and the backward pass (backpropagation).
- Activation functions (ReLU, tanh, sigmoid) and loss functions.
- Gradient descent and the general training loop.
- Overfitting and basic regularization strategies (dropout, data augmentation).
- CNNs as an example of structured, parameter-sharing networks.
- Basic familiarity with sequences: text, time series, audio as ordered collections of elements.

0 Learning Objectives

After studying these notes, you should be able to:

- explain why feedforward networks (MLPs and CNNs) are poorly suited for sequential data, and describe the inductive bias that RNNs introduce;
- describe the recurrent computation in a vanilla RNN and write out the state update and output equations;
- explain the vanishing and exploding gradient problems and why they are especially severe in deep unrolled RNNs;
- describe the LSTM architecture, identify its four gates, and explain how the cell state mitigates the vanishing gradient problem;
- describe the GRU architecture and explain how it simplifies the LSTM while retaining its key advantages;
- identify common sequence modeling tasks (many-to-one, one-to-many, many-to-many) and map them to appropriate RNN configurations;
- implement a character-level language model and a sequence classifier using PyTorch's `nn.RNN`, `nn.LSTM`, and `nn.GRU` modules;
- diagnose common RNN training problems and apply appropriate fixes.

1 Introduction

1.1 Why Not Just Use an MLP or CNN for Sequences?

MLPs and CNNs process inputs of a fixed size and treat each input independently of all others. An MLP applied to a sentence, for example, must receive a sentence of a predetermined length; it has no mechanism to remember information from a previous sentence or from earlier parts of the same sentence. This is a fundamental mismatch with the structure of sequential data.

Variable-length inputs. Real-world sequences e.g., sentences, audio recordings, time series of financial prices, DNA strands) vary in length. A fixed-size architecture cannot directly accommodate this variability without ad-hoc preprocessing (truncation, zero-padding) that may discard or corrupt important information.

Order matters. In a sentence, the meaning of a word depends on the words that preceded it. In a time series, the value at time t depends on the history of values before t . A standard MLP treats all input positions independently; it has no built-in notion of temporal order.

Long-range dependencies. Understanding the sentence “The animal didn’t cross the street because *it* was too tired” requires knowing that *it* refers to *the animal*, not *the street*. Such dependencies can span many time steps, requiring the model to maintain and retrieve information over long horizons.

Recurrent Neural Network

A *Recurrent Neural Network* (RNN) is a neural network with *directed cycles* in its connectivity. At each time step, the network receives an input and updates a hidden state that summarizes all relevant information from the current and all previous inputs. This hidden state is passed forward to the next time step, giving the network an explicit memory mechanism. RNNs are the standard architecture for processing sequences of variable length.

1.2 The Sequential Inductive Bias

Just as CNNs encode the assumption that nearby pixels are more likely to be related (local connectivity and parameter sharing), RNNs encode the assumption that the current observation depends on its history, and that this dependency is captured by a fixed-size summary called the hidden state. Formally:

- *Temporal parameter sharing.* The same weight matrices are applied at every time step. This is the temporal analogue of the spatial parameter sharing in CNNs: a pattern detector that recognizes a word as a verb should work whether the word appears at position 3 or position 15 in the sentence.
- *Markov-like state compression.* Rather than storing the full history of inputs, the RNN compresses it into a fixed-size hidden state vector $\mathbf{h}_t$. Ideally, $\mathbf{h}_t$ retains whatever is needed to predict future outputs and discards the rest.

- *Variable-length processing.* Because the same recurrent step is applied for as many time steps as the sequence is long, a single model handles sequences of any length without modification.

Historical context. Early recurrent networks were proposed in the 1980s [Rumelhart et al., 1986, Elman, 1990]. The problem of training them through long sequences was identified and analyzed by Hochreiter [1991] and Bengio et al. [1994]. The Long Short-Term Memory (LSTM) architecture, introduced by Hochreiter and Schmidhuber [1997], became the dominant solution for over two decades and remains widely used. Gated Recurrent Units (GRUs) [Cho et al., 2014] provided a simpler alternative. Since 2017, Transformer models [Vaswani et al., 2017] have largely replaced RNNs for many sequence tasks, but RNNs remain relevant for streaming inference, on-device deployment, and as a conceptual foundation for understanding sequence modeling.

1.3 Conceptual Roadmap

Before diving into the details, it helps to see the whole story at once. Each section of these notes addresses one step in the following progression.

The RNN Storyline

- Step 1.** *The Problem.* MLPs and CNNs cannot handle variable-length, ordered sequences or learn long-range dependencies. (Section 1)
- Step 2.** *The Core Idea.* Introduce a hidden state as a running memory and share weight matrices across time. (Section 2)
- Step 3.** *The First Attempt: Vanilla RNN.* Simple and elegant, but training collapses for long sequences due to vanishing and exploding gradients. (Section 3)
- Step 4.** *The Breakthrough: LSTM.* A cell state highway with learnable gates creates a near-unobstructed gradient path, fixing the vanishing gradient problem in practice. (Section 4)
- Step 5.** *The Simplification: GRU.* Merges cell state and hidden state, reducing four gates to two, yielding 75% of LSTM parameters with comparable performance on most tasks. (Section 5)
- Step 6.** *The Landscape.* Practical configurations (many-to-one, encoder-decoder, bidirectional), implementation, debugging, and the transition to Transformers. (Sections 6–9)

Entry Ticket 1

Think about what you already know about MLPs and CNNs before reading about recurrence. Before class, run the following PyTorch snippet and bring your observations:

```
1 import torch
2 import torch.nn as nn
3
4 rnn = nn.RNN(input_size=10, hidden_size=20, batch_first=True)
5 x = torch.randn(4, 15, 10) # (batch, seq_len, features)
```

```

6 out, h_n = rnn(x)
7 print(f"Output shape: {out.shape}")
8 print(f"Hidden shape: {h_n.shape}")

```

1. Record the output shapes. Why does `out` have 15 time steps but `h_n` only 1?
2. Add `num_layers=3` to the `nn.RNN` call. What happens to the shape of `h_n`? Why?
3. Suppose you want to classify whether a movie review is positive or negative. The reviews vary in length from 20 to 500 words. Why is it difficult to apply a standard MLP directly to word sequences of variable length?
4. In a CNN, the same kernel is applied at every spatial location. What would the temporal analogue of this be for a sequence model? Why is this a useful property?

2 The Vanilla RNN

2.1 State Update Equations

The simplest RNN processes a sequence $(\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T)$ one element at a time. At each step t , it combines the current input $\mathbf{x}_t \in \mathbb{R}^d$ with the previous hidden state $\mathbf{h}_{t-1} \in \mathbb{R}^h$ to produce a new hidden state $\mathbf{h}_t \in \mathbb{R}^h$, and optionally an output $\mathbf{y}_t$.

Vanilla RNN Update Equations

$$\mathbf{h}_t = \tanh(\mathbf{W}_{hh} \mathbf{h}_{t-1} + \mathbf{W}_{xh} \mathbf{x}_t + \mathbf{b}_h), \quad (1)$$

$$\mathbf{y}_t = \mathbf{W}_{hy} \mathbf{h}_t + \mathbf{b}_y. \quad (2)$$

where $\mathbf{W}_{hh} \in \mathbb{R}^{h \times h}$ is the recurrent weight matrix, $\mathbf{W}_{xh} \in \mathbb{R}^{h \times d}$ is the input weight matrix, $\mathbf{W}_{hy} \in \mathbb{R}^{o \times h}$ is the output weight matrix, and $\mathbf{b}_h, \mathbf{b}_y$ are bias vectors. The initial hidden state $\mathbf{h}_0$ is typically set to the zero vector.

The key structural feature is that $\mathbf{W}_{hh}$, $\mathbf{W}_{xh}$, and $\mathbf{W}_{hy}$ are *shared across all time steps*. The network at time t is not a different network from the one at time $t+1$; it is the same network applied again with a new input and the updated hidden state.

Why Tanh?

The tanh activation is used in the vanilla RNN because it maps its input to $(-1, 1)$, keeping hidden state values bounded. Using ReLU in the recurrent connection tends to produce instability: if the hidden state grows without bound, the next step amplifies it further, causing numerical overflow. Tanh provides a soft saturation that prevents this. The LSTM and GRU architectures address this more systematically with gating mechanisms (Section 4).

Myth: The RNN “Remembers” Like a Database

The hidden state $\mathbf{h}_t$ is a compressed, lossy summary of the past, not a perfect memory buffer. It has a fixed size (e.g., 256 numbers) regardless of how long the sequence is. For a 1,000-step sequence, the network must encode all task-relevant information from 1,000 inputs into a single fixed-size vector. This is an extreme information bottleneck, and it explains why even LSTMs struggle with very long sequences (e.g., documents of 10,000 words). Think of it as a running notepad of limited size: the network must learn to write only what matters and overwrite what does not.

2.2 Unrolling Through Time

The recurrence in Eq. 1 is compact, but it can also be misleading at first glance. The equation shows only one recurrent step, as if the RNN were a small module with a loop. However, when the model processes a complete sequence, that same module is applied repeatedly: once for $\mathbf{x}_1$, then again for $\mathbf{x}_2$, and so on until $\mathbf{x}_T$.

To make this repeated computation easier to visualize, we often *unroll* (or *unfold*) the RNN through time. Unrolling means replacing the recurrent loop by an explicit chain of identical copies of the same computation, one copy for each time step. This does not create new parameters. It is only a different way of drawing the same model.

In the unrolled view, the RNN looks like a feedforward network with depth T , where T is the sequence length. The important difference from an ordinary feedforward network is that each layer in this chain uses the same weight matrices. The matrices $\mathbf{W}_{xh}$, $\mathbf{W}_{hh}$, and $\mathbf{W}_{hy}$ are shared across time. Thus, the copy of the RNN used at time step 1 is not a different network from the copy used at time step 4; it is the same network applied again to a different input and a different previous hidden state.

Figure 1 illustrates this idea for a sequence of length 4. The gold boxes at the bottom represent the input sequence:

$$\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \mathbf{x}_4.$$

At each time step t , the model receives the current input $\mathbf{x}_t$ and combines it with the previous hidden state $\mathbf{h}_{t-1}$. This produces a new hidden state $\mathbf{h}_t$, represented by the purple boxes. The hidden states are connected horizontally because they carry information forward through the sequence. For example, $\mathbf{h}_3$ is influenced by $\mathbf{x}_3$, but also indirectly by $\mathbf{x}_1$ and $\mathbf{x}_2$ through $\mathbf{h}_2$.

The green boxes at the top represent the outputs $\mathbf{y}_t$. Depending on the task, we may use all outputs or only the final one. For example, in a many-to-many task such as part-of-speech tagging, the model may produce one output for each input token. In a many-to-one task such as sentiment classification, we may ignore the intermediate outputs and use only the final hidden state or final output as a summary of the whole sequence.

This unrolled representation is useful for two reasons. First, it clarifies how the hidden state acts as a running summary of the sequence. Each hidden state contains information from the current input and, through the previous hidden state, from earlier inputs. Second, it explains why training RNNs can be difficult. During backpropagation, gradients must flow backward through this entire chain of repeated computations. Therefore, an

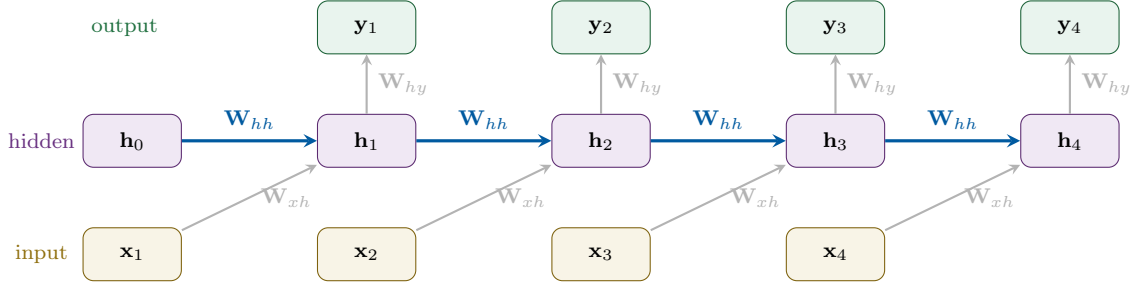


Figure 1: An RNN unrolled through 4 time steps. The bottom row shows the input sequence, the middle row shows the hidden states, and the top row shows the outputs. At time step t , the hidden state $\mathbf{h}_t$ is computed from two sources: the current input $\mathbf{x}_t$, through $\mathbf{W}_{xh}$, and the previous hidden state $\mathbf{h}_{t-1}$, through $\mathbf{W}_{hh}$. The output $\mathbf{y}_t$ is then computed from $\mathbf{h}_t$ through $\mathbf{W}_{hy}$. Although the diagram shows four copies of the computation, these copies do not have separate parameters: the same matrices $\mathbf{W}_{xh}$, $\mathbf{W}_{hh}$, and $\mathbf{W}_{hy}$ are reused at every time step.

RNN processing a long sequence behaves like a very deep network, which is precisely what leads to the vanishing and exploding gradient problems discussed in Section 3.2.

2.3 A Worked Example

Consider a minimal RNN with input dimension $d = 2$, hidden dimension $h = 2$, and the following (hand-chosen) parameters:

$$\mathbf{W}_{xh} = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix}, \quad \mathbf{W}_{hh} = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix}, \quad \mathbf{b}_h = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad \mathbf{h}_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

For the input sequence $\mathbf{x}_1 = [1, 0]^\top$ and $\mathbf{x}_2 = [0, 1]^\top$:

Step 1:

$$\mathbf{h}_1 = \tanh(\mathbf{W}_{hh}\mathbf{h}_0 + \mathbf{W}_{xh}\mathbf{x}_1) = \tanh\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0.5 \\ 0 \end{bmatrix}\right) = \tanh\left(\begin{bmatrix} 0.5 \\ 0 \end{bmatrix}\right) \approx \begin{bmatrix} 0.462 \\ 0 \end{bmatrix}.$$

Step 2:

$$\mathbf{h}_2 = \tanh(\mathbf{W}_{hh}\mathbf{h}_1 + \mathbf{W}_{xh}\mathbf{x}_2) = \tanh\left(\begin{bmatrix} 0.231 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 0.5 \end{bmatrix}\right) = \tanh\left(\begin{bmatrix} 0.231 \\ 0.5 \end{bmatrix}\right) \approx \begin{bmatrix} 0.227 \\ 0.462 \end{bmatrix}.$$

After two steps, $\mathbf{h}_2$ encodes information from both inputs: the first component reflects the history of $\mathbf{x}_1$, and the second reflects $\mathbf{x}_2$. In practice, the weight matrices are learned so that the hidden state captures task-relevant information.

2.4 Counting Parameters

A vanilla RNN with input dimension d , hidden dimension h , and output dimension o has:

$$\underbrace{h \times d}_{\mathbf{W}_{xh}} + \underbrace{h \times h}_{\mathbf{W}_{hh}} + \underbrace{h}_{\mathbf{b}_h} + \underbrace{o \times h}_{\mathbf{W}_{hy}} + \underbrace{o}_{\mathbf{b}_y}$$

learnable parameters. Crucially, this count does not depend on the sequence length T ; the same parameters handle sequences of any length.

Entry Ticket 2

Work through the following before reading about training difficulties.

1. A vanilla RNN has $d = 50$, $h = 128$, $o = 10$. How many learnable parameters does it have? How does this compare to an MLP with one hidden layer of 128 neurons and the same input/output sizes?
2. The unrolled RNN for a sequence of length $T = 100$ looks like a feedforward network of depth 100. What consequence does this have for gradient computation during training?
3. If the hidden state $\mathbf{h}_t$ has dimension $h = 256$, what is the shape of $\mathbf{W}_{hh}$? Why might a very large spectral radius of $\mathbf{W}_{hh}$ be problematic for gradient flow?

3 Training RNNs: Backpropagation Through Time

3.1 The BPTT Algorithm

Training an RNN follows the same principle as training any neural network: compute the loss, run backpropagation, and update parameters. Because the network is unrolled through time, the backpropagation algorithm applied to RNNs is called *Backpropagation Through Time* (BPTT). At a high level, BPTT is identical to standard backpropagation applied to the unrolled computational graph: gradients flow backward through each time step.

For a sequence of length T with per-step losses ℓ_t , the total loss is:

$$\mathcal{L} = \sum_{t=1}^T \ell_t.$$

The gradient of $\mathcal{L}$ with respect to the recurrent weight matrix $\mathbf{W}_{hh}$ accumulates contributions from every time step:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hh}} = \sum_{t=1}^T \frac{\partial \ell_t}{\partial \mathbf{W}_{hh}}.$$

Each term $\partial \ell_t / \partial \mathbf{W}_{hh}$ requires backpropagating through all t steps up to that point, producing a product of Jacobians.

3.2 The Vanishing and Exploding Gradient Problems

The gradient of the loss at time step t with respect to the hidden state at an earlier time step $k < t$ involves the product of $t - k$ Jacobian matrices:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} = \prod_{i=k+1}^t \frac{\partial \mathbf{h}_i}{\partial \mathbf{h}_{i-1}} = \prod_{i=k+1}^t \text{diag}(\tanh'(\mathbf{u}_i)) \mathbf{W}_{hh}^\top,$$

where $\mathbf{u}_i = \mathbf{W}_{hh}\mathbf{h}_{i-1} + \mathbf{W}_{xh}\mathbf{x}_i + \mathbf{b}_h$ is the pre-activation at step i . This is a product of $(t - k)$ matrices, each involving both the recurrent weight matrix and the derivative of the activation function.

If the spectral radius (largest singular value) of each Jacobian is consistently less than 1, the product shrinks exponentially with $(t - k)$, causing vanishing gradients: gradients from distant past steps become negligibly small, preventing the network from learning long-range dependencies.

If the spectral radius is consistently greater than 1, the product grows exponentially, causing exploding gradients: gradients become numerically unstable, causing divergent updates.

Vanishing and Exploding Gradients in RNNs

For a vanilla RNN, the gradient $\partial \mathbf{h}_t / \partial \mathbf{h}_k$ decays exponentially with the distance $(t - k)$ if the spectral radius of $\mathbf{W}_{hh}$ is less than 1, and grows exponentially if it is greater than 1. This means that the vanilla RNN effectively cannot learn dependencies spanning more than roughly 10–20 time steps in practice. These problems are the primary motivation for the LSTM and GRU architectures.

3.3 Truncated BPTT and Gradient Clipping

Two practical techniques are used to mitigate training instability in RNNs.

Truncated BPTT. Rather than backpropagating through the entire sequence of length T , training is performed by breaking the sequence into fixed-length chunks and backpropagating through at most k steps within each chunk. The hidden state is still carried forward between chunks, so the forward pass retains long-range context, but gradients are only backpropagated within each chunk. This significantly reduces memory cost and stabilizes training at the expense of some gradient accuracy over very long ranges.

Gradient clipping. Exploding gradients are addressed by *gradient clipping*: if the global norm of the gradient vector exceeds a threshold τ , the gradient is rescaled so that its norm equals τ :

$$\mathbf{g} \leftarrow \frac{\tau}{\|\mathbf{g}\|} \mathbf{g} \quad \text{if } \|\mathbf{g}\| > \tau.$$

This prevents individual updates from taking excessively large steps, stabilizing training without fundamentally changing the gradient direction.

Gradient Clipping in PyTorch

In PyTorch, gradient clipping is applied after `loss.backward()` and before `optimizer.step()`:

```
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
```

A common value for `max_norm` is 1.0 or 5.0. Clipping norms exceeding these values prevents parameter updates from becoming numerically unstable during training on long sequences.

Entry Ticket 3

Reflect on the training difficulties before reading about gated architectures.

1. Explain in your own words why the gradient of the loss at step T with respect to the hidden state at step 1 can become extremely small in a vanilla RNN. What property of the weight matrix determines whether the gradient vanishes or explodes?
2. Gradient clipping addresses exploding gradients but not vanishing gradients. Why? What is needed to address vanishing gradients more fundamentally?
3. What is the practical consequence of vanishing gradients for a task like sentiment analysis on long movie reviews?

4 Long Short-Term Memory (LSTM)

4.1 Motivation and Architecture Overview

The LSTM was introduced by [Hochreiter and Schmidhuber \[1997\]](#) specifically to address the vanishing gradient problem. Its central insight is to introduce a *cell state* $\mathbf{c}_t$, which is a separate memory vector that runs through the sequence with only mild, additive interactions. The LSTM controls what information is written to, read from, and erased from this cell state using *gating mechanisms*.

At each time step, the LSTM computes four vectors using the current input $\mathbf{x}_t$ and the previous hidden state $\mathbf{h}_{t-1}$: a *forget gate* $\mathbf{f}_t$, an *input gate* $\mathbf{i}_t$, a *cell gate* $\mathbf{g}_t$ (also called the candidate cell state), and an *output gate* $\mathbf{o}_t$. These gates use the sigmoid activation $\sigma(\cdot)$, whose output lies in $(0, 1)$ and acts as a soft on/off switch.

4.2 LSTM Equations

LSTM Update Equations

Given input $\mathbf{x}_t \in \mathbb{R}^d$, previous hidden state $\mathbf{h}_{t-1} \in \mathbb{R}^h$, and previous cell state $\mathbf{c}_{t-1} \in \mathbb{R}^h$:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f [\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_f) \quad (\text{forget gate}) \quad (3)$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i [\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_i) \quad (\text{input gate}) \quad (4)$$

$$\mathbf{g}_t = \tanh(\mathbf{W}_g [\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_g) \quad (\text{cell gate / candidate}) \quad (5)$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o [\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_o) \quad (\text{output gate}) \quad (6)$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{g}_t \quad (\text{cell state update}) \quad (7)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad (\text{hidden state}) \quad (8)$$

where $[\mathbf{h}_{t-1}; \mathbf{x}_t] \in \mathbb{R}^{h+d}$ denotes concatenation, $\odot$ denotes the element-wise (Hadamard) product, and σ is the sigmoid function.

Notation: Concatenated Weight Matrices

In the LSTM equations above, $\mathbf{W}_f \in \mathbb{R}^{h \times (h+d)}$ operates on the concatenated vector $[\mathbf{h}_{t-1}; \mathbf{x}_t]$. This is equivalent to writing $\mathbf{W}_{fh} \mathbf{h}_{t-1} + \mathbf{W}_{fx} \mathbf{x}_t$ with separate weight matrices; the concatenated form is simply more compact and matches how PyTorch implements it internally.

Why Tanh Appears Twice in the LSTM

The tanh activation serves two distinct purposes in the LSTM:

- In the candidate cell gate $\mathbf{g}_t$ (Eq. 5): The output lies in $(-1, 1)$, so $\mathbf{g}_t$ can either reinforce (positive values) or suppress (negative values) what is currently stored in $\mathbf{c}_{t-1}$. The input gate $\mathbf{i}_t \in (0, 1)$ then controls how much of this reinforcement or suppression is actually written. Using a linear activation here would allow unbounded growth of the cell state over time.
- In the hidden state output $\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$ (Eq. 8): The cell state $\mathbf{c}_t$ is updated additively and can accumulate large values over many steps. The tanh squashes it back into $(-1, 1)$ before it is gated by $\mathbf{o}_t$, ensuring that the hidden state exposed to subsequent layers remains in a well-behaved range. Without this squashing, the hidden state passed to the next layer could grow very large even when the output gate is small.

4.3 Understanding the Gates

Each gate controls a specific aspect of information flow through the cell state:

- **Forget gate** $\mathbf{f}_t \in (0, 1)^h$: decides what fraction of the previous cell state $\mathbf{c}_{t-1}$ to retain. A value near 1 means “keep this memory”; near 0 means “erase it.” For instance, when processing a new sentence, the forget gate may learn to reset the cell state to avoid contamination from the previous sentence.
- **Input gate** $\mathbf{i}_t \in (0, 1)^h$: decides which dimensions of the candidate cell state $\mathbf{g}_t$ to write into the cell. It acts as a filter on new information.
- **Cell gate** $\mathbf{g}_t \in (-1, 1)^h$: the candidate update to the cell state, computed using tanh. It represents what new information would be written if the input gate were fully open.
- **Output gate** $\mathbf{o}_t \in (0, 1)^h$: decides which dimensions of the cell state are exposed as the hidden state $\mathbf{h}_t$, which is the output of the LSTM at step t .

Why the Cell State Helps: The Gradient Highway

Consider how the gradient of the loss flows back through the cell state. From Eq. 7, the partial derivative of $\mathbf{c}_t$ with respect to $\mathbf{c}_{t-1}$ is:

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} = \text{diag}(\mathbf{f}_t).$$

This is a diagonal matrix (element-wise multiplication by $\mathbf{f}_t$), not a full matrix multiplication by $\mathbf{W}_{hh}^\top$ as in the vanilla RNN. The practical consequence is profound:

over k steps, the gradient flows through a product of diagonal matrices, not a product of full weight matrices. There is no large, potentially unstable weight matrix amplifying or squashing the gradient at every step. When the forget gate is near 1 ($\mathbf{f}_t \approx \mathbf{1}$), each diagonal matrix is close to the identity, and the gradient propagates with almost no attenuation over hundreds of steps. The forget gate thus acts as a *soft, learnable switch* for gradient flow through each individual cell state dimension. This is the principal reason LSTMs can learn dependencies that span hundreds of time steps while vanilla RNNs cannot.

4.4 Counting LSTM Parameters

One reason the LSTM is more powerful than the vanilla RNN is that it does not compute just one hidden-state update. Instead, at every time step it computes *four* different intermediate quantities: the forget gate $\mathbf{f}_t$, the input gate $\mathbf{i}_t$, the candidate cell update $\mathbf{g}_t$, and the output gate $\mathbf{o}_t$. Each of these quantities is obtained from the same input pair (the previous hidden state $\mathbf{h}_{t-1}$ and the current input $\mathbf{x}_t$), but with its own learnable affine transformation.

Using the compact notation introduced earlier, each gate receives the concatenated vector $[\mathbf{h}_{t-1}; \mathbf{x}_t] \in \mathbb{R}^{h+d}$. Therefore, each gate has:

- one weight matrix of shape $h \times (h + d)$, and
- one bias vector of shape h .

Since there are four such gate computations, the total number of learnable parameters in a single LSTM layer is

$$\text{Parameters} = 4 \times (h(h + d) + h) = 4h(h + d + 1).$$

This expression is worth interpreting carefully:

- the factor 4 appears because the LSTM computes four gate-related affine transformations;
- the term $h(h + d)$ comes from the weight matrix for each gate;
- the final $+h$ comes from the bias vector for each gate.

Example. Suppose the input dimension is $d = 64$ and the hidden dimension is $h = 128$. Then a single LSTM layer has

$$4 \times 128 \times (128 + 64 + 1) = 4 \times 128 \times 193 = 98,816$$

learnable parameters.

This is substantially larger than the recurrent core of a vanilla RNN with the same input and hidden dimensions, whose parameter count is only $h(h + d + 1)$. In that sense, the LSTM is roughly four times more expensive in parameters than a vanilla RNN. This

additional cost is the price paid for the gating mechanisms that give the LSTM much finer control over memory.

It is useful to connect this parameter count to the internal structure of the LSTM cell. Figure 2 shows the flow of information at one time step.

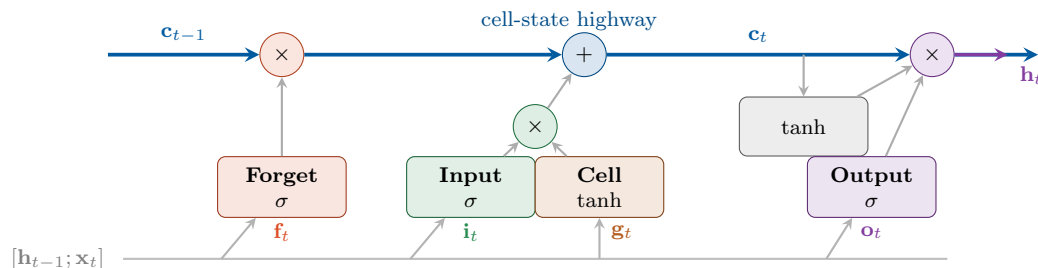


Figure 2: Internal structure of an LSTM cell at time step t . The top horizontal path is the cell-state pathway. First, the forget gate $\mathbf{f}_t$ decides how much of the previous cell state $\mathbf{c}_{t-1}$ should be retained. Next, the input gate $\mathbf{i}_t$ and the candidate update $\mathbf{g}_t$ work together to write new information into the memory. Their contribution is added to the retained part of $\mathbf{c}_{t-1}$ to form the new cell state $\mathbf{c}_t$. Finally, the output gate $\mathbf{o}_t$ determines how much of the transformed cell state $\tanh(\mathbf{c}_t)$ is exposed as the hidden state $\mathbf{h}_t$. This additive cell-state update is the key structural reason why LSTMs support gradient flow much better than vanilla RNNs.

The figure can be read from left to right in four conceptual steps:

1. *Keep or erase old memory.* The forget gate $\mathbf{f}_t$ scales the previous cell state $\mathbf{c}_{t-1}$ element-wise. If some component of $\mathbf{f}_t$ is close to 1, that part of the memory is kept; if it is close to 0, that part is erased.
2. *Propose new memory content.* The cell gate $\mathbf{g}_t$ produces candidate values that could be written into memory.
3. *Control how much new information is written.* The input gate $\mathbf{i}_t$ filters the candidate values $\mathbf{g}_t$. The product $\mathbf{i}_t \odot \mathbf{g}_t$ is the actual update written into the cell state.
4. *Expose part of the memory as output.* After the new cell state $\mathbf{c}_t$ is formed, the output gate $\mathbf{o}_t$ decides which parts of $\tanh(\mathbf{c}_t)$ become visible as the hidden state $\mathbf{h}_t$.

This interpretation is important because it shows that the LSTM is not merely a larger RNN. It is an architecture in which different gates specialize in different memory operations: forgetting, writing, and exposing information. The larger parameter count is therefore not arbitrary; it reflects this more structured control over information flow.

Common Misconception

It is tempting to say that the LSTM *solves* the vanishing gradient problem once and for all. That statement is too strong. The LSTM creates a much easier path for gradients to flow through the cell state, but this path is still modulated by the forget gate. If the forget gate repeatedly outputs values close to 0, information is intentionally discarded and gradients can still decay. Thus, the LSTM does not guarantee perfect long-range memory in every situation; rather, it gives the network

a mechanism that makes long-range memory *possible* and learnable in practice. For this reason, practitioners often initialize the forget-gate bias to a positive value (for example, 1.0), encouraging the network to begin training with the memory pathway relatively open.

5 Gated Recurrent Unit (GRU)

5.1 Simplifying the LSTM

The GRU was introduced by [Cho et al. \[2014\]](#) as a simpler alternative to the LSTM. It merges the cell state and the hidden state into a single vector and uses only two gates: an *update gate* $\mathbf{z}_t$ and a *reset gate* $\mathbf{r}_t$. The GRU has three weight matrices instead of four, retaining 75% of the parameter count of an equivalent LSTM while achieving comparable performance on most tasks.

5.2 GRU Equations

GRU Update Equations

Given input $\mathbf{x}_t$ and previous hidden state $\mathbf{h}_{t-1}$:

$$\mathbf{z}_t = \sigma(\mathbf{W}_z [\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_z) \quad (\text{update gate}) \quad (9)$$

$$\mathbf{r}_t = \sigma(\mathbf{W}_r [\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_r) \quad (\text{reset gate}) \quad (10)$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h [\mathbf{r}_t \odot \mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_h) \quad (\text{candidate hidden state}) \quad (11)$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t. \quad (\text{hidden state update}) \quad (12)$$

5.3 Interpreting the GRU Gates

- Update gate $\mathbf{z}_t$: controls how much of the previous hidden state $\mathbf{h}_{t-1}$ is carried forward versus how much is replaced by the new candidate $\tilde{\mathbf{h}}_t$. When $\mathbf{z}_t \approx 0$, the hidden state is almost entirely replaced. When $\mathbf{z}_t \approx 1$, the hidden state is almost entirely preserved.
- Reset gate $\mathbf{r}_t$: controls how much of the previous hidden state is used in computing the candidate $\tilde{\mathbf{h}}_t$. When $\mathbf{r}_t \approx 0$, the candidate is computed almost entirely from the current input, ignoring past state, effectively resetting the memory.

Structural Analogy between GRU and LSTM

Comparing the GRU hidden state update (Eq. 12) with the LSTM cell state update (Eq. 7) reveals a perfect structural analogy:

LSTM cell state	$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{g}_t$
GRU hidden state	$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t$

In both cases, the update is a *convex interpolation* between the old state and a new candidate: keep a fraction of the past, write a fraction of the new. The differences

are:

- The LSTM's forget gate $\mathbf{f}_t$ is analogous to $(1 - \mathbf{z}_t)$ in the GRU: both control how much of the past to retain.
- The LSTM's input gate $\mathbf{i}_t$ is analogous to $\mathbf{z}_t$: both control how much new information to write.
- Critically, the LSTM allows $\mathbf{f}_t$ and $\mathbf{i}_t$ to be *independent*, so the network can simultaneously retain old information and write new information even when $\mathbf{f}_t + \mathbf{i}_t \neq \mathbf{1}$. The GRU enforces $(1 - \mathbf{z}_t) + \mathbf{z}_t = \mathbf{1}$, a hard coupling that reduces expressivity slightly but also reduces parameter count.
- The GRU's reset gate $\mathbf{r}_t$ has no direct LSTM counterpart; it controls how much the previous hidden state influences the candidate $\tilde{\mathbf{h}}_t$ itself, a form of intra-step selective forgetting not present in the LSTM.

GRU vs. LSTM: When to Use Which

On many standard benchmarks (language modeling, machine translation, speech recognition), GRUs and LSTMs achieve comparable performance. GRUs have 75% of the parameters of an equivalent LSTM and train faster. LSTMs may have an edge on tasks requiring very precise long-range memory control due to the explicit separation of cell state and hidden state. In practice, the default recommendation is to start with an LSTM; switch to a GRU if training speed or parameter count is a constraint. If both underperform, the architecture or data is likely the bottleneck.

Table 1 summarizes the differences between the three architectures.

Table 1: Comparison of vanilla RNN, LSTM, and GRU architectures.

Property	Vanilla RNN	LSTM	GRU
Number of gates	0	4 (f, i, g, o)	2 (z, r)
Separate cell state	No	Yes ($\mathbf{c}_t$)	No
Parameters (per layer)	$h(h + d + 1) + oh$	$4h(h + d + 1)$	$3h(h + d + 1)$
Relative parameter count	baseline	$\approx 4 \times$ RNN	75% of LSTM
Long-range memory	Poor	Good	Good
Training speed	Fastest	Slowest	Middle
Typical use case	Short sequences; baselines	Default for most tasks	Efficient alternative to LSTM

Entry Ticket 4

Work through the LSTM and GRU before reading about sequence tasks.

1. In the LSTM cell state update $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{g}_t$, the gradient $\partial \mathbf{c}_t / \partial \mathbf{c}_{t-1} = \text{diag}(\mathbf{f}_t)$. Explain in your own words why a diagonal matrix is preferable to

the full $\mathbf{W}_{hh}^\top$ in the vanilla RNN for gradient flow over long sequences.

2. Using the structural analogy identified in Section 5.3: if the LSTM's forget and input gates are independent ($\mathbf{f}_t + \mathbf{i}_t$ can differ from $\mathbf{1}$), what kind of behavior can an LSTM express that a GRU cannot?
3. An LSTM with $d = 128$ and $h = 256$ has how many parameters? A GRU with the same dimensions has how many? What is the ratio, and does it match the stated 75%?

6 Sequence Modeling Tasks and RNN Configurations

6.1 Taxonomy of Sequence Tasks

RNNs can be configured to address a wide variety of input-output patterns. The choice of configuration depends on the task.

Sequence Task Taxonomy

- Many-to-one: the entire input sequence is mapped to a single output. Examples: sentiment classification, document classification, time-series anomaly labeling.
- One-to-many: a single input (or a context vector) is expanded into an output sequence. Examples: image captioning, music generation from a genre label.
- Many-to-many (synchronized): each input step produces an output step. Examples: named entity recognition (word-to-label), video frame classification.
- Many-to-many (encoder-decoder): an input sequence of one length is mapped to an output sequence of a different length. Examples: machine translation, speech recognition, text summarization.

Figure 3 illustrates these configurations schematically.

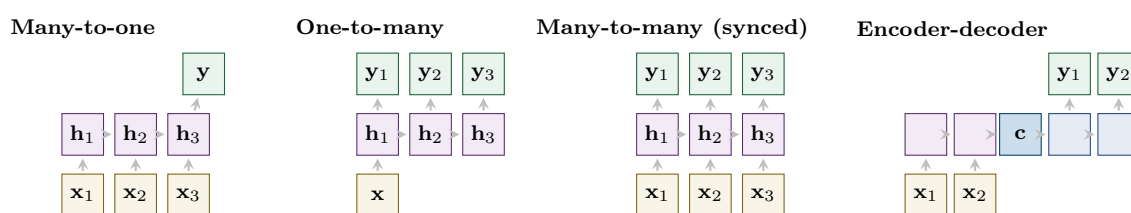


Figure 3: Four common RNN configurations for sequence tasks. Boxes represent input ($\mathbf{x}$, gold), hidden state ($\mathbf{h}$, purple), and output ($\mathbf{y}$, green) at each time step. In the encoder-decoder configuration, the encoder compresses the input sequence into a context vector $\mathbf{c}$, which initializes the decoder.

6.2 Bidirectional RNNs

A standard (forward) RNN processes the sequence from left to right. For tasks where the full sequence is available and future context is relevant (e.g., named entity recognition, machine translation encoding), a bidirectional RNN runs two separate RNNs: one

forward and one backward over the sequence. The hidden states from both directions are concatenated at each step:

$$\vec{\mathbf{h}}_t = \text{RNN}_{\text{fwd}}(\mathbf{x}_t, \vec{\mathbf{h}}_{t-1}), \quad \overleftarrow{\mathbf{h}}_t = \text{RNN}_{\text{bwd}}(\mathbf{x}_t, \overleftarrow{\mathbf{h}}_{t+1}), \quad \mathbf{h}_t = [\vec{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t].$$

Bidirectional RNNs Cannot Be Used for Generation

Bidirectional RNNs require the full sequence to be available before processing can begin (the backward pass needs the end of the sequence first). They are therefore unsuitable for autoregressive generation tasks (language modeling, text generation) where the output must be produced one token at a time without seeing future inputs. They are well-suited for encoding tasks (classification, translation) where the entire input is available at once.

6.3 Stacked (Deep) RNNs

Multiple RNN layers can be stacked, where the output sequence of one layer becomes the input sequence of the next. Stacked RNNs learn hierarchical temporal representations: lower layers capture local, short-range patterns; higher layers capture more abstract, longer-range patterns. A stacked LSTM with two layers is a common default architecture.

Table 2: Common RNN-based architectures and their typical use cases.

Architecture	Task type	Example applications
Unidirectional RNN/LSTM	Many-to-one	Sentiment analysis, time-series prediction
Bidirectional LSTM	Many-to-many (sync)	NER, POS tagging, machine translation encoding
Encoder-decoder LSTM	Many-to-many (async)	Machine translation, summarization, Q&A
Stacked LSTM	Many-to-one or seq2seq	Complex language understanding tasks
Language model RNN	Many-to-many (sync)	Text generation, speech recognition

Entry Ticket 5

Consider the following tasks and map each to a sequence configuration.

1. A model that reads a sequence of daily stock prices and outputs a single prediction of whether the stock will rise or fall tomorrow. Which configuration is this?
2. A model that takes a sentence in English and produces a sentence in Portuguese. The two sentences may have different numbers of words. Which configuration is this, and why can a synchronized many-to-many model not be used directly?

3. Why would you use a bidirectional LSTM for named entity recognition (tagging each word in a sentence) but not for generating a sentence word-by-word?

7 Implementation in PyTorch

7.1 RNN Modules in PyTorch

PyTorch provides ready-to-use modules for vanilla RNNs, LSTMs, and GRUs: `nn.RNN`, `nn.LSTM`, and `nn.GRU`. All three share the same interface. The key arguments in such interface are `input_size` (d), `hidden_size` (h), `num_layers` (number of stacked layers), `batch_first` (whether the batch dimension is first or second), and `bidirectional`.

For an LSTM, the forward method returns both the sequence of hidden states *and* the final $(\mathbf{h}_T, \mathbf{c}_T)$ tuple. For many-to-one tasks, only the final hidden state $\mathbf{h}_T$ is needed.

7.2 Sequence Classifier

Listing 1 implements a many-to-one sequence classifier (e.g., sentiment analysis) using an LSTM in PyTorch.

```

1 import torch
2 import torch.nn as nn
3
4 class LSTMClassifier(nn.Module):
5     def __init__(self, vocab_size, embed_dim, hidden_dim,
6                 num_layers, num_classes, dropout=0.3):
7         super().__init__()
8         # Embedding layer: integer token IDs -> dense vectors
9         self.embedding = nn.Embedding(vocab_size, embed_dim,
10                                     padding_idx=0)
11         # Stacked bidirectional LSTM
12         self.lstm = nn.LSTM(
13             input_size=embed_dim,
14             hidden_size=hidden_dim,
15             num_layers=num_layers,
16             batch_first=True,          # input: (batch, seq_len,
17                                     features)
18             dropout=dropout if num_layers > 1 else 0.0,
19             bidirectional=True
20         )
21         self.dropout = nn.Dropout(dropout)
22         # Factor 2 because bidirectional doubles the hidden size
23         self.fc = nn.Linear(hidden_dim * 2, num_classes)
24
25     def forward(self, x):
26         # x: (batch, seq_len) integer token IDs
27         embedded = self.dropout(self.embedding(x))
28         # output: (batch, seq_len, 2*hidden_dim)
29         # h_n:      (num_layers*2, batch, hidden_dim)

```

```

29         output, (h_n, c_n) = self.lstm(embedded)
30         # Concatenate final forward and backward hidden states
31         # h_n[-2]: last forward layer; h_n[-1]: last backward layer
32         h_final = torch.cat([h_n[-2], h_n[-1]], dim=1)
33         h_final = self.dropout(h_final)
34         return self.fc(h_final)                # (batch, num_classes)
35
36 model      = LSTMClassifier(vocab_size=10000, embed_dim=128,
37                             hidden_dim=256,  num_layers=2,
38                             num_classes=2)
39 criterion = nn.CrossEntropyLoss()
40 optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

```

Listing 1: LSTM-based sequence classifier in PyTorch (many-to-one).

The Embedding Layer: Efficient One-Hot Lookup

Discrete tokens (characters, words, subwords) cannot be fed directly into a neural network as integers. An `nn.Embedding(vocab_size, embed_dim)` layer converts each integer token ID to a learned dense vector of dimension `embed_dim`.

Mathematically, `nn.Embedding` is identical to `nn.Linear(vocab_size, embed_dim, bias=False)`: if you one-hot encode a token ID into a vector of length `vocab_size` (a vector of all zeros except a single 1 at the token's index), multiplying it by the `vocab_size × embed_dim` weight matrix simply selects the row corresponding to that token. The embedding layer performs this row lookup *directly* from the integer index, without ever constructing the large sparse one-hot vector — which is why it is far more memory-efficient. The embedding weights are trained jointly with the rest of the model via backpropagation.

7.3 Character-Level Language Model

A language model assigns a probability to each token in a sequence given the preceding tokens. At each step, the model reads the current token, updates the hidden state, and predicts a probability distribution over the vocabulary for the next token. This is a many-to-many (synchronized) task. Listing 2 implements a character-level language model using `nn.GRU`.

```

1 class CharLM(nn.Module):
2     def __init__(self, vocab_size, embed_dim, hidden_dim,
3                 num_layers):
4         super().__init__()
5         self.embedding = nn.Embedding(vocab_size, embed_dim)
6         self.gru = nn.GRU(
7             input_size=embed_dim,
8             hidden_size=hidden_dim,
9             num_layers=num_layers,
10            batch_first=True,
11        )
12        self.fc = nn.Linear(hidden_dim, vocab_size) # predict next
              char

```

```

12
13     def forward(self, x, h=None):
14         # x: (batch, seq_len) integer character IDs
15         emb = self.embedding(x)                # (batch,
16         seq_len, embed)
17         out, h_n = self.gru(emb, h)            # out: (batch,
18         seq_len, hidden)
19         logits = self.fc(out)                  # (batch,
20         seq_len, vocab_size)
21         return logits, h_n
22
23 def train_step(model, batch, criterion, optimizer):
24     # batch: (batch_size, seq_len+1); inputs are[:-1], targets are
25     # [1:]
26     inputs, targets = batch[:, :-1], batch[:, 1:]
27     optimizer.zero_grad()
28     logits, _ = model(inputs)
29     # logits: (batch, seq_len, vocab); reshape for CrossEntropyLoss
30     loss = criterion(logits.reshape(-1, logits.size(-1)),
31                       targets.reshape(-1))
32     loss.backward()
33     # Clip gradients to prevent exploding gradient problem
34     torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm
35     =1.0)
36     optimizer.step()
37     return loss.item()

```

Listing 2: Character-level language model using a GRU.

7.4 Training Loop with Gradient Clipping

Listing 3 shows the general training and evaluation loop for a sequence classifier, including gradient clipping and the correct use of `model.train()` and `model.eval()`.

```

1 NUM_EPOCHS = 20
2 CLIP_NORM = 1.0
3
4 for epoch in range(NUM_EPOCHS):
5     # --- Training phase ---
6     model.train()
7     train_loss = 0.0
8     for sequences, labels in train_loader:
9         optimizer.zero_grad()
10        logits = model(sequences)
11        loss = criterion(logits, labels)
12        loss.backward()
13        # Prevent exploding gradients
14        torch.nn.utils.clip_grad_norm_(model.parameters(),
15        CLIP_NORM)
16        optimizer.step()
17        train_loss += loss.item()

```

```

17
18     # --- Evaluation phase ---
19     model.eval()
20     correct, total = 0, 0
21     with torch.no_grad():
22         for sequences, labels in val_loader:
23             preds = model(sequences).argmax(dim=1)
24             correct += (preds == labels).sum().item()
25             total += labels.size(0)
26     val_acc = correct / total
27     print(f"Epoch {epoch+1:02d} | "
28           f"Train Loss: {train_loss/len(train_loader):.4f} | "
29           f"Val Acc: {val_acc:.4f}")

```

Listing 3: Training and evaluation loop for an RNN-based sequence classifier.

Stateful vs. Stateless RNN Training

In Listing 3, the hidden state is reset to zero at the start of each batch (the default behavior when no initial state is provided). This is called *stateless* training, and is appropriate when sequences in the batch are independent (e.g., individual sentences). For tasks where the input is a single very long sequence processed in chunks (e.g., a long document or a continuous audio stream), the hidden state can be carried forward between chunks (*stateful* training), requiring explicit management of the hidden state across batches.

8 Common Pitfalls and Debugging RNNs

8.1 First Sanity Checks

Before optimizing hyperparameters, verify the following:

- *Shape consistency.* For `batch_first=True`, the input to an RNN layer must be shaped `(batch, seq_len, input_size)`. Use `print(x.shape)` at each step during development.
- *Can the model overfit one batch?* Train for many epochs on a single small batch. The training loss should reach near-zero; if not, the architecture or loss function has a fundamental error.
- *Gradient norms.* If the model fails to learn at all, inspect gradient norms. Extremely small (vanishing) or NaN (exploded) values indicate training instability.
- *Padding and masking.* If sequences in a batch have different lengths and are zero-padded, ensure the loss is not computed over padding tokens. Use the class `nn.CrossEntropyLoss(ignore_index=0)`.
- *Train and evaluation modes.* Dropout is active during `model.train()` and disabled during `model.eval()`. Always call the correct mode before the appropriate phase.

8.2 Debugging Checklist

Table 3 summarizes common symptoms and their likely causes when training RNNs.

Table 3: Common symptoms, likely causes, and fixes when training RNNs.

Symptom	Likely cause	Fix
Loss is NaN from step 1	Exploding gradients; LR too large	Add gradient clipping; reduce LR
Loss does not decrease at all	Vanishing gradients in vanilla RNN	Switch to LSTM or GRU
Loss decreases then plateaus	Hidden dimension too small; underfitting	Increase hidden size or add layers
Good training loss, poor val loss	Overfitting; too little data	Dropout, reduce model size, more data
Accuracy stuck near $1/K$	Incorrect label handling; wrong loss	Verify integer labels; check <code>CrossEntropyLoss</code>
<code>RuntimeError: shape mismatch</code>	<code>batch_first</code> inconsistency; wrong <code>input_size</code>	Check tensor shapes at each layer
Slow training	Long sequences; large hidden size	Truncated BPTT; reduce hidden size; use GPU
Padding tokens affect loss	No masking applied	Use <code>ignore_index</code> in loss
Model performs worse at test time	<code>model.train()</code> not switched off	Always call <code>model.eval()</code> before evaluation

Entry Ticket 6

An LSTM language model is trained on a corpus of news articles. The training perplexity decreases steadily, but the validation perplexity is much higher and stops improving after a few epochs.

1. What phenomenon is occurring? What is perplexity, and how is it related to the cross-entropy loss?
2. Describe three interventions specific to sequence models that might reduce the gap between training and validation perplexity.
3. During text generation with this model, you feed one character at a time and pass the hidden state forward between steps. Why must you call `model.eval()` before generation? What could go wrong if you forget?

9 Connection to the Broader Deep Learning Landscape

9.1 RNNs as Structured MLPs

Just as a CNN can be understood as an MLP with constraints that reflect the grid structure of images (local connectivity, spatial parameter sharing), an RNN can be understood as an MLP with constraints that reflect the sequential structure of time series and language:

- *Temporal parameter sharing*: the same weight matrices $\mathbf{W}_{xh}$ and $\mathbf{W}_{hh}$ are applied at every time step. This is the sequential analogue of a convolutional kernel applied at every spatial position.
- *State compression*: the hidden state $\mathbf{h}_t$ summarizes all past inputs into a fixed-size vector, analogous to how feature maps in a CNN summarize spatial structure up to a given depth.
- *Variable-length handling*: because the same computation is applied for as many steps as the sequence is long, the model is not tied to a fixed input size.

The unification is important: CNNs, RNNs, and MLPs are all instances of a more general principle — neural networks with structured constraints that match the symmetries and dependencies in the data. Choosing the right architecture is fundamentally about choosing the right inductive bias.

9.2 The Transformer and the Limits of RNNs

Despite their success, RNNs have two significant limitations that became increasingly apparent as models scaled up:

Sequential computation bottleneck. Because $\mathbf{h}_t$ depends on $\mathbf{h}_{t-1}$, the hidden states must be computed one at a time. This prevents parallelization across the time dimension during training, making RNNs slow on modern GPU hardware.

Long-range information bottleneck. Even with LSTMs and GRUs, information from early time steps must pass through many intermediate hidden states to influence the output at a late step. The fixed-size hidden state is a fundamental compression bottleneck.

The Transformer architecture [Vaswani et al., 2017] addresses both problems by replacing recurrence with *self-attention*: every position in the sequence can directly attend to every other position, enabling full parallelization and direct long-range connections. Transformers are now the dominant architecture for natural language processing, audio, and increasingly for vision. However, RNNs remain important for streaming inference on edge devices and for autoregressive tasks where inputs arrive one step at a time.

RNNs Beyond Text

RNNs are not limited to text. Vanilla RNNs and LSTMs have been successfully applied to time-series forecasting (weather, finance, sensor data), speech recognition and synthesis, music generation, genomic sequence modeling, and reinforcement learning (as the memory component of agents). The sequence modeling abstraction is extremely general: any process that produces a sequence of observations over time can potentially benefit from an RNN-based model.

Entry Ticket 7

Reflect on the connections between RNNs and the broader course content.

1. A CNN exploits the spatial structure of images through parameter sharing; an RNN exploits the temporal structure of sequences through the same principle. In one or two sentences, state what is shared and what is different between these two forms of parameter sharing.
2. The Transformer uses self-attention instead of recurrence. What two problems of RNNs does this solve? What does it give up compared to an RNN in terms of inference on a stream of incoming data?
3. The progression of this course has been: MLP $\rightarrow$ CNN $\rightarrow$ RNN. What is the common theme across all three? What does each architecture add relative to the previous one?

10 Final Remarks

10.1 Summary of Key Concepts

The following points summarize the essential content of these notes:

- Feedforward networks cannot handle variable-length sequences or model temporal dependencies. RNNs address this by maintaining a *hidden state* updated at each time step — but this state is a *compressed, lossy summary* of the past, not a perfect memory.
- A vanilla RNN updates its hidden state as $\mathbf{h}_t = \tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h)$, with the same weight matrices shared across all time steps.
- Training RNNs uses *BPTT*. The gradient flows through a product of Jacobians, causing *vanishing gradients* (long sequences) or *exploding gradients* (large spectral radius). Gradient clipping mitigates the latter.
- The *LSTM* introduces a cell state $\mathbf{c}_t$ with an additive update $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{g}_t$. The gradient $\partial \mathbf{c}_t / \partial \mathbf{c}_{t-1} = \text{diag}(\mathbf{f}_t)$ is a diagonal matrix (not a full matrix multiplication), providing a near-unobstructed gradient highway through long sequences.
- The *GRU* uses a single interpolation $\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t$, structurally analogous to the LSTM cell state update but with coupled forget and input gates. GRUs have 75% of the parameters of an equivalent LSTM.
- RNNs support diverse configurations: many-to-one, one-to-many, synchronized many-to-many, and encoder-decoder. Bidirectional RNNs exploit full-sequence context but cannot be used for generation.
- In PyTorch, `nn.LSTM` and `nn.GRU` share the same interface. Key considerations are `batch_first`, proper hidden state initialization, gradient clipping, and padding/-masking.
- The *Transformer* supersedes RNNs for large-scale NLP by replacing recurrence with self-attention, enabling parallelization and direct long-range connections. RNNs remain relevant for streaming and edge deployment.

10.2 Recommended Next Steps

These notes prepare the student for the following topics in the deep learning module:

MLP → CNNs → **RNNs** → **Attention & Transformers** → Generative Models

Key Takeaways

- RNNs exploit sequential structure through temporal parameter sharing and a hidden state, exactly as CNNs exploit spatial structure — but the hidden state is a *lossy compression*, not a lossless record.
- The vanilla RNN suffers from vanishing/exploding gradients. The LSTM solves this through an *additive*, gate-controlled cell state whose gradient involves only diagonal (not full matrix) multiplication. The GRU provides a simpler, structurally analogous solution with 75% of the parameters.
- Bidirectional and stacked variants extend both architectures; the encoder-decoder pattern handles input and output sequences of different lengths.
- PyTorch's `nn.LSTM`/`nn.GRU` are straightforward to use; the most common bugs are shape mismatches, forgotten gradient clipping, and missing `model.eval()` calls.
- The Transformer has largely superseded RNNs for large-scale tasks, but understanding RNNs thoroughly is the conceptual prerequisite for understanding attention and Transformers.

You are designing a model to transcribe a live audio stream for a voice assistant. Audio arrives continuously as a real-time stream; there is no defined start or end.

1. You must choose between a bidirectional LSTM and a unidirectional LSTM. Which would you choose for this streaming task? Justify your answer by explaining why the other is unsuitable.
2. Describe the state management problem you would need to solve to apply your chosen model to an infinite stream. How would you decide when to reset the hidden state?
3. Your training data consists of long recorded audio clips (up to 30 minutes, sampled at 16 kHz). Unrolling the full sequence would require millions of BPTT steps. Which training technique would you use, and what is the trade-off it introduces?
4. During training on these long clips, you observe NaN losses after a few hundred steps. What is the likely cause, and what single line of PyTorch code would you add to fix it?

10 References

Yoshua Bengio, Patrice Simard, and Paolo Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2):157–166, 1994.

Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using

- RNN encoder–decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734. Association for Computational Linguistics, 2014.
- Jeffrey L. Elman. Finding structure in time. *Cognitive Science*, 14(2):179–211, 1990.
- Sepp Hochreiter. Untersuchungen zu dynamischen neuronalen netzen. Technical report, Technische Universität München, 1991. Diploma thesis, supervised by Jürgen Schmidhuber.
- Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986. doi: 10.1038/323533a0.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 5998–6008, 2017. URL <https://papers.nips.cc/paper/7181-attention-is-all-you-need.pdf>.